

Master Thesis

Tracing the coronary arteries in 3D computed tomography scans

Weekly Report 10

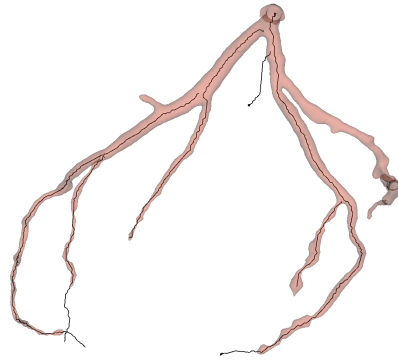
Malgorzata Korona. s223322
Mateusz Baldachowski. s230241

April 10, 2025

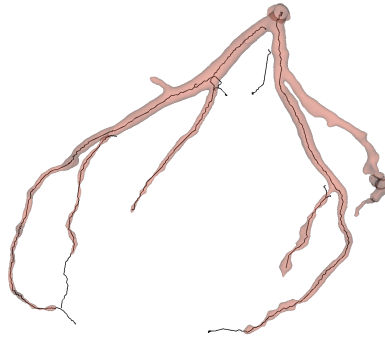
1 This week's progress

1.1 Radius based branch spawning

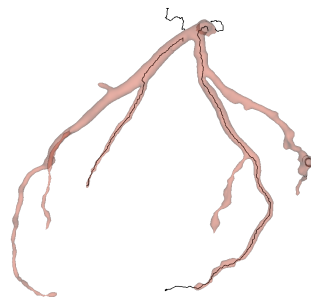
We implemented branch spawning based on the radius. We experimented with looking for starting points of new branches in area located from the detected branch point by $k \cdot r_{\text{est}}$ for $k = 1, 2, 3$. Below are the results of tracing the coronary artery tree for 3 chosen images from the test set.



(a) $k = 1$

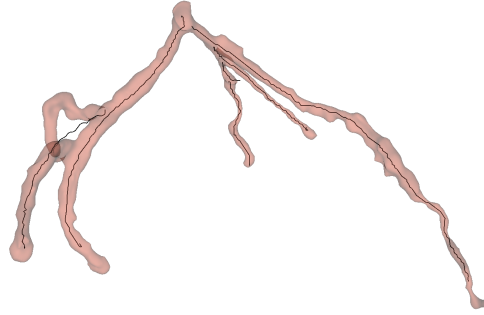


(b) $k = 2$

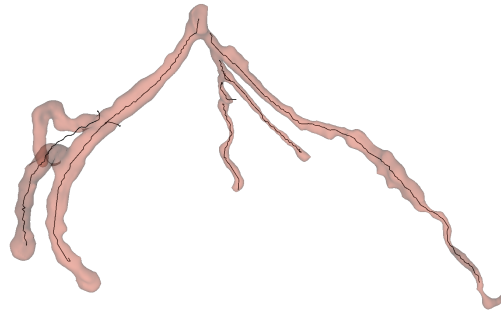


(c) $k = 3$

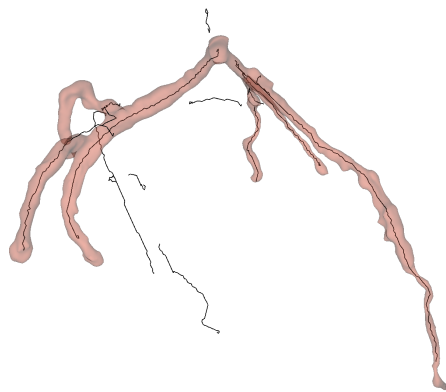
Figure 1: Detected coronary tree in image 800 for different values of scale k .



(a) $k = 1$

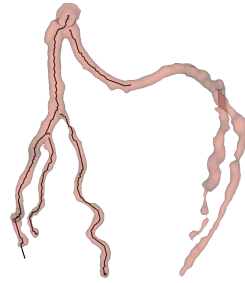


(b) $k = 2$

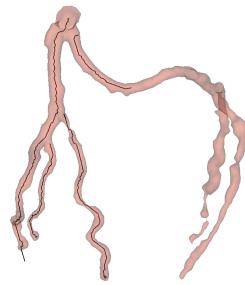


(c) $k = 3$

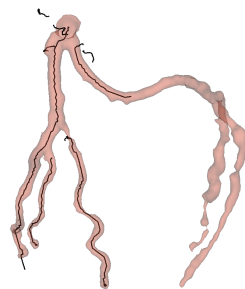
Figure 2: Detected coronary tree in image 804 for different values of scale k .



(a) $k = 1$



(b) $k = 2$



(c) $k = 3$

Figure 3: Detected coronary tree in image 817 for different values of scale k .

For $k = 3$ the results are of course completely crazy and unreasonable, for $k = 1$ and $k = 2$ they look similar, but let's see what happen when we zoom in a bit:

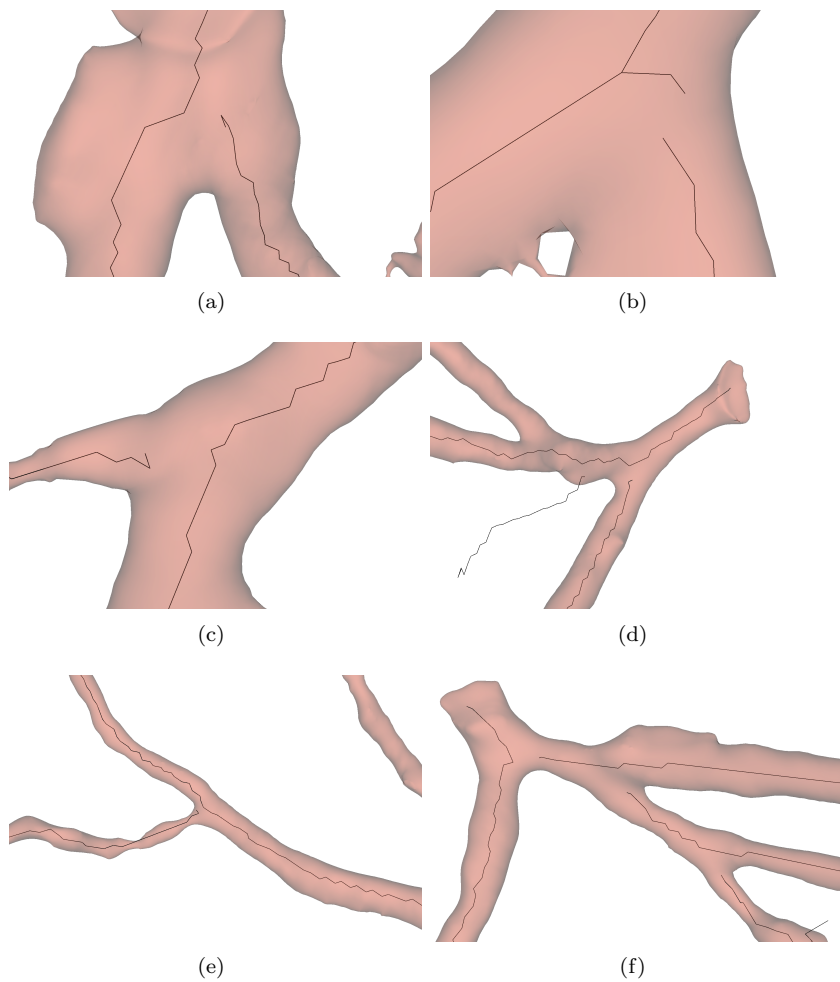


Figure 4: Zoom in on branch points for $k = 1$

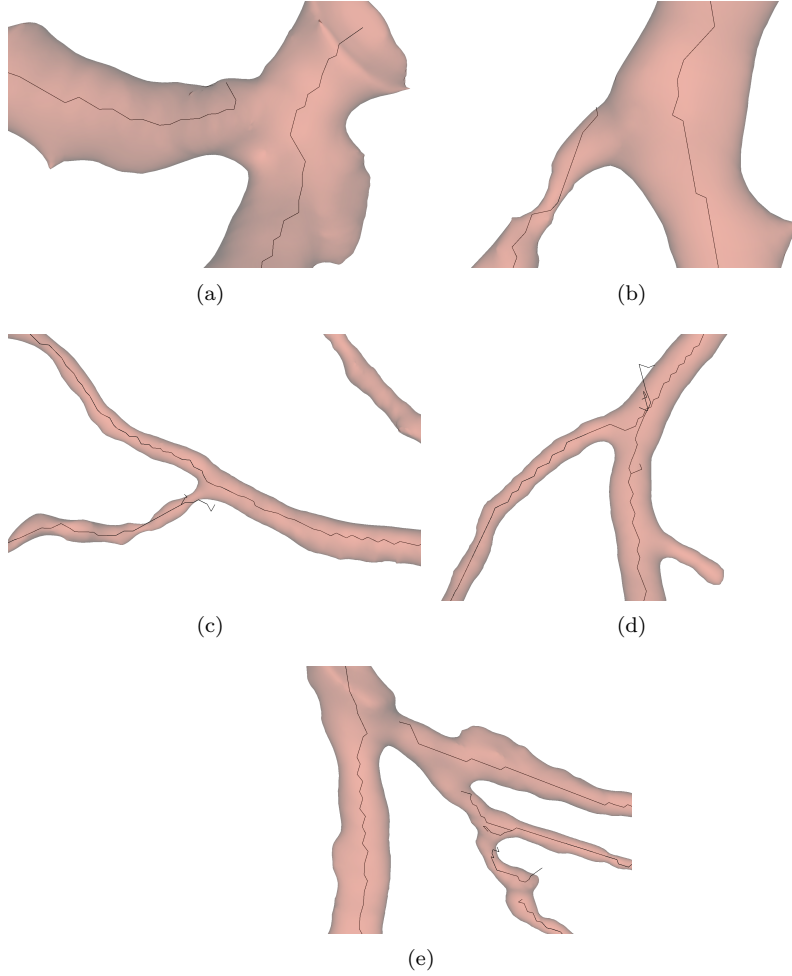


Figure 5: Zoom in on branch points for $k = 2$

For $k = 2$ tracing of new branches usually starts a bit off the true centerline, sometimes it wanders around the branch point for several step, and only later it converges to where it should be. For $k = 1$ we can see that there is sometimes one step where the tracer initially goes in the wrong direction, but it still stays close to the centerline from the very beginning and it doesn't wander around it. It seems that with appropriate post-processing this could work :) It would also be nice to have some method of evaluating this approach on all images at once, without having to look at every one of them.

1.2 Data

1.2.1 Data preprocessing

Each image and the corresponding segmentation has ben processed so that it can serve as an input to the relevant models, or to be used in a tracing algo-

rithm. During data processing, each scan has been associated with the following information:

- starting and end points of the coronary artery tree,
- centerlines,
- branch points locations.

The centerlines are not directly available in the ImageCAS dataset - only the coronary artery meshes are provided there. Hence, the centerlines were extracted given the available segmentations in a following process:

1. Both left and right coronary arteries arise from the aorta above the left and right cusps of the aortic valve, correspondingly[1]. Thus, the starting points were found as an overlap between the segmentations of aorta and the dilation of segmentations of coronary arteries. Center of the overlap was then defined as a starting point.
2. End points in each coronary artery tree were found with VTK function `vtkDijkstraGraphGeodesicPath` that uses Dijkstra's algorithm to find shortest geodesic paths between given points - in our case between the starting point and points on the surface of coronary artery segmentation. End points were defined as surface points that correspond to the local maxima of geodesic distances in small regions.
3. Having coronary artery surfaces, start and end points, `vmrkcenterlines` method from VMTK library was used to extract the centerlines. This method outputs a set of branches, where each branch consists of set of points, that together form a coronary artery tree. Points where different branches merge together serve as branch points (bifurcations). Additionally, each point i is assigned a scalar value corresponding to the radius r_i of a maximum inscribed sphere. The results are shown in figure 6(a). Since the algorithm is not precise in areas where arteries were narrow (e.g. in stenoses), the segmentation was artificially dilated in such regions to ensure proper centerline extraction. This resulted in overestimated spheres (figure 6(b)). Nevertheless, the results were accurate enough for the goal of branch spawning.



Figure 6: Example of LCA segmentation (in pink) together with maximum inscribed spheres (in yellow). (a) shows the whole coronary tree, whereas (b) shows an overestimation of the spheres due to the narrowing of artery structure.

After centerline extraction, each point is assigned a value corresponding to a distance from a given point to a starting point in millimeters. Thus, the distances are monotonically increasing, reflecting the flow of blood through arteries. An example of such heatmap is presented in figure 7.

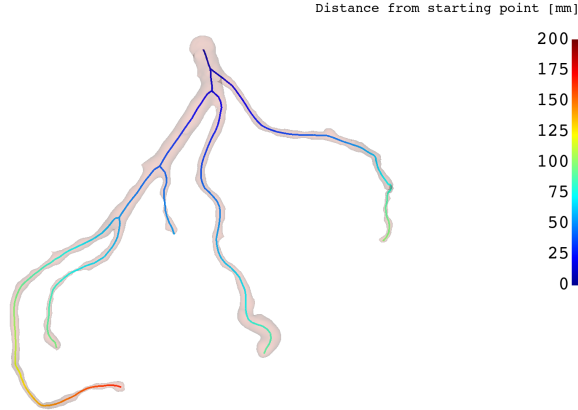


Figure 7: Example of LCA centerline with distances heatmap together with coronary artery segmentation.

Additionally, for the branch point detection model use case, we assign a likelihood of each point on the centerline being a branch point. We define an influence radius of 5 millimeters. If a given point lies outside of influence radius from a ground truth branch point, the likelihood assigned to it is equal to zero. Otherwise, we calculate the probability density v_i for a point i having distance to the branch point d_i using a Gaussian distribution, i.e. $d_i \sim \mathcal{N}(0, 2^2)$, and $v_i = f(d_i) = \frac{1}{\sqrt{2\pi \cdot 2^2}} e^{-\frac{(d_i^2)}{2 \cdot 2^2}}$. A heatmap of branch point likelihoods is presented in figure 8

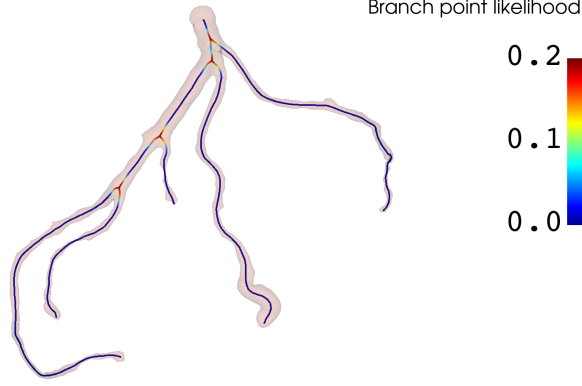


Figure 8: Example of LCA centerline with likelihoods of points being branch points.

1.2.2 Image processing

Having the preprocessed data, during the training of the models, as well as in the tracing algorithm, we perform several steps of image processing.

Original image scans are non-isotropic, meaning that they have different spacing in different axes. This can cause problems when training convolutional neural networks, since the same areas of an image can contain slightly different information in each scan, making it difficult for the model to learn appropriate features from the dataset and generalize well to the new data. To avoid potential issues, all scans are resampled to the isotropic pixel spacing with voxel spacing equal to 0.5mm.

Additionally, we perform HU augmentation on the images. Iodinated CT contrast corresponds to HU range of 100-600 [2]. In order to capture not only the lumen, but also the calcified plaque and surrounding soft tissue, we defined our HU range as -200 to 1200. At the same time, having in mind that the reference values might differ between individuals, we defined a minimum HU range as $[-800, -200]$ and maximum HU range as $[1200, 2000]$. For each sample, we took a random value from minimum and maximum HU ranges and clipped all other voxel intensities to those values. Then, we normalized the intensities to $[-1, 1]$ scale according to the formula:

$$I_{\text{after}} = (I_{\text{before}} - HU_{\text{min}}) \frac{HU_{\text{max}} - HU_{\text{min}}}{I_{\text{max}}^{\text{new}} - I_{\text{min}}^{\text{new}}} + I_{\text{min}}^{\text{new}} \quad (1)$$

where I_{before} , I_{after} are images before and after normalization, HU_{min} and HU_{max} are HU values sampled from minimum and maximum HU ranges, $I_{\text{min}}^{\text{new}}$ and $I_{\text{max}}^{\text{new}}$ are new minimum and maximum intensity values equal to -1 and 1 , correspondingly.

1.3 Tracing model

Tracing coronary artery is essentially done by sequential movements of an agent starting from the intersection of the artery with the aorta. To facilitate that, the agent must know which direction to follow in every possible state where state is defined as a local image patch centered around the agent. In each step the agent is allowed to follow any of 26 directions representing adjacent voxels on a 3x3x3 cube.

1.3.1 Reward function

To be able to access the quality of the step from one point to the other a concept of reward function is introduced. The reward function is based on distances to the closest point on the artery’s centerline and along it. It encourages taking actions which keep the agent close to the centerline but also follow it along. The second condition is needed as otherwise the tracker would not have an incentive to move further once it reached the centerline. The proposed reward function, assessing the quality of each move, is adopted from [?] approach and is calculated with this equation:

$$R = \begin{cases} \omega\alpha(D_{\text{to}}(p) - D_{\text{to}}(p_{\text{next}})) & \text{if } D_{\text{to}}(p_{\text{next}}) > \tau \\ \omega(\alpha(D_{\text{to}}(p) - D_{\text{to}}(p_{\text{next}})) + (D_{\text{along}}(p_{\text{next}}) - D_{\text{along}}(p))) & \text{otherwise} \end{cases}$$

As an agent moves from p to p_{next} the distances to the centerline and along the centerline are calculated for both points to evaluate reward. Those distances are feeded to the reward function. If a distance to the centerline in the new state exceeded threshold denoted by τ , only the difference in the distance to the centerline between two points is considered. If however the distance to the centerline in the new state is within the limit, the reward function takes into account change in distance along the centerline as well. Additional two parameters are used $\alpha = 2$, which in the latter cases puts more weight on proximity to the centerline rather than distance along it, and $w = 1/\text{steplength}$ to account for bias towards selecting diagonal directions. The distances two and along the centerline are calculated in the following way:

$$\begin{aligned} D_{\text{along}}(p) = v_k &= \frac{\|g_k - g_{k-1}\|_2}{d_{\text{total}}} \cdot v_{k+1} + \frac{\|g_k - g_{k+1}\|_2}{d_{\text{total}}} \cdot v_{k-1}, \\ D_{\text{to}}(p) &= \|p - g_k\|_2 \\ d_{\text{total}} &= \|g_{k-1} - g_{k+1}\|_2. \end{aligned}$$

where g_k denotes the closest point on the centerline for p , and v_k represents cumulative distance along the centerline from the root of the artery. The D_{along} uses an interpolation to rectify the possibility of the closest point on the centerline laying between the two previously defined points on the centerline.

Important to note, that during tracing true distances to the centerline are unknown so the above calculation are used to calculate the ground truth reward

which the convolutional neural network is trained to estimate. Therefore it is the estimated reward values that are used by the agent to choose the direction.

TODO: image with reward function

1.3.2 Convolutional neural network

Our approach at evaluating reward value utilizes convolutional neural network (CNN). The network input is a 41 x 41 x 41 image patch, corresponding to a 20.5 mm x 20.5 mm x 20.5 mm area. The input is first passed through four blocks of small convolutional networks, each consisting of two convolutional layers followed by batch normalization and wrapped up with ReLU activation function. It is then pooled and passed to through three fully connected layers, each followed by ReLU activation function, to finally output the likelihood of each of the 26 possible directions to be the best choice.

TODO: Adjust that to reflect the final model.

TODO: image with architecture of the network.

1.3.3 Training setup

During the training, we feed the CNN defined above with image patches extracted for each sample from the memory. We defined a memory capacity as 25000, and with 450 training images we get 55 samples per scan in each epoch. Samples in the memory are shuffled so that we do not end up with a network biased towards some specific coronary artery. We perform the training with batches of size 64, and we run through the memory 10 times per epoch before we resample the data and fill the memory again. The training runs for 50 epochs.

We use cross-entropy loss to calculate the loss for each output from the network and Adam optimizer to update the weights of our CNN.

Table 1 summarizes current settings used during the training step:

Training parameter	Value
number of epochs	50
runs through memory	10
batch size	64
learning rate	10^{-4}
weight decay	10^{-3}

Table 1: Overview of current training setup

To evaluate the training progress we calculate the average training loss per epoch as a total loss divided by number of batches in this epoch and runs through memory.

1.3.4 Validation setup

Validation is performed every 5th epoch. The model is evaluated on a 150 validation samples, and for each sample 64 tracer steps are performed. In each step, we find a random point laying on a ground truth centerline, and an image patch centered in this point is propagated through the network that outputs the likelihood of each possible direction.

Apart from evaluating model based on validation loss and accuracy we additionally introduced average reward and weighted accuracy. Average reward is a total reward for each predicted direction, calculated as described in section 1.3.1 divided by number of validation sample. Weighted accuracy however is an extension of regular accuracy in which each direction is weighted based on how close it is to the true best direction. The weights are obtained by taking a dot product between each direction and the ground truth direction. Negative weights, for vectors with angle above 90 degrees, are forced to 0 to avoid negative reward values.

TODO: Double check normalization and adjust text.

2 Project status

This week we implemented branch spawning based on the estimated radius, trying to multiply radius by different values to see what gives best results, and it seems that it helps with the issue of the tracer going around the true coronary artery. We only looked visually at a few images, so it would be nice to come up with some kind of a method to evaluate if this approach actually works or not taking all scans into account.

We also focused on some writing: we didn't really write new content, but we organized most of the stuff we had written so far and rewrote them to have more master thesis style text. We also started gathering everything we have in a thesis draft.

3 Discussion points

1. Should we go into more details in the data preprocessing section? E.g. describe the algorithms for extracting the centerline or calculating distance from the starting points more in detail. Is what we have right now enough?
2. Was there any postprocessing other than smoothing the centerlines done by Theodora? Joining the branches? Removing odd short centerline fragments?
3. Should data section in thesis only include ImageCAS dataset description? Is data processing part of method?

4 Plan for the next weeks

- Easter Break
- Finish checking radius based branch spawning
- Start working on coronary artery presence model
- Do some structuring/cleaning of the code we have. Starts being a bit problematic :)

References

- [1] Achilles J. Pappano and Withrow Gil Wier. 11 - coronary circulation. In Achilles J. Pappano and Withrow Gil Wier, editors, *Cardiovascular Physiology (Tenth Edition)*, pages 223–236. Elsevier, Philadelphia, tenth edition edition, 2013.
- [2] M.H. Lev and R.G. Gonzalez. 17 - ct angiography and ct perfusion imaging. In Arthur W. Toga and John C. Mazziotta, editors, *Brain Mapping: The Methods (Second Edition)*, pages 427–484. Academic Press, San Diego, second edition edition, 2002.